

DSA Revision Notes

Topic 11 — Backtracking & Trie

Backtracking template · Pruning · Subsets · Permutations · Combinations
N-Queens · Sudoku · Word Search II · Trie from scratch · 25+ C++ examples

12

Sections

25+

C++ Examples

8

Tree Diagrams

22+

LeetCode Problems

TOC Table of Contents**1. Backtracking — Core Theory**

- Choose–Explore–Unchoose template · Decision tree
- Pruning strategies · Time complexity analysis

2. Subsets (Power Set)

- Iterative approach · Backtracking approach
- Subsets II (with duplicates) · Subsets of size k

3. Permutations

- Permutations I (distinct) · Permutations II (duplicates)
- Next Permutation · Permutation Sequence

4. Combinations

- Combinations (nCk) · Combination Sum I, II, III
- Letter combinations of phone number

5. Palindrome Partitioning

- All palindrome partitions · Min cuts (DP + BT)

6. N-Queens & N-Queens II

- Constraint modelling · Diagonal checks · Optimisations

7. Sudoku Solver (LC 37)

- Constraint propagation · Row/col/box bitmask

8. Word Search on Grid

- Word Search I (DFS) · Word Search II (Trie + DFS)

9. Trie — Core Theory & Implementation

- Trie node structure · Insert · Search · StartsWith
- Time/space analysis · Compressed Trie

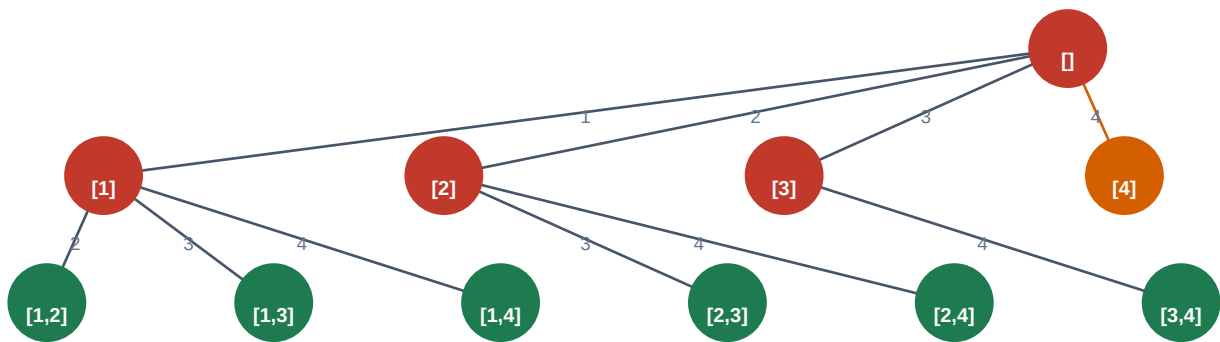
10. Trie — Classic Problems

- Implement Trie · Replace Words · Map Sum Pairs
- Search Suggestions System · Design Add-Search Words

11. Trie — Advanced Applications

- Maximum XOR of Two Numbers · XOR Trie
- Word Search II (Trie + Backtracking combined)

12. Complexity Cheat Sheet & LeetCode Map



Combinations $C(4,2)$: green=answer, orange=pruned (no more choices after 4)

Decision tree for combinations of size 2 from [1,2,3,4]. Each level = one element added.

2 Subsets (Power Set)

2.1 Subsets I — All Subsets of Distinct Elements (LC 78)

Generate all 2^n subsets of a set of distinct integers. Backtracking: at each index, either include or exclude the element. Alternative: iterate index **start** to avoid duplicates — each element appears at most once per subset.

```
C++
1 // LC 78 - Subsets
2 vector<vector<int>> subsets(vector<int>& nums) {
3     vector<vector<int>> result;
4     vector<int> current;
5     function<void(int)> bt = [&](int start) {
6         result.push_back(current); // every state is a valid subset
7         for (int i = start; i < (int)nums.size(); i++) {
8             current.push_back(nums[i]); // CHOOSE
9             bt(i + 1); // EXPLORE (i+1: no reuse)
10            current.pop_back(); // UNCHOOSE
11        }
12    };
13    bt(0);
14    return result;
15 }
16 // Time: O(n * 2^n) Space: O(n) recursion depth
17
18 // Iterative bit-mask approach
19 vector<vector<int>> subsetsBit(vector<int>& nums) {
20     int n = nums.size();
21     vector<vector<int>> result;
22     for (int mask = 0; mask < (1<<n); mask++) {
23         vector<int> sub;
24         for (int i = 0; i < n; i++)
25             if (mask & (1<<i)) sub.push_back(nums[i]);
26         result.push_back(sub);
27     }
28     return result; // Time: O(n * 2^n) Space: O(1) extra
29 }
```

2.2 Subsets II — With Duplicates (LC 90)

When the input has duplicates, sort first, then skip duplicates at the same recursion level. Key: skip if $i > \text{start}$ && $\text{nums}[i] == \text{nums}[i-1]$ — this means the same value was already chosen at this level, so we'd generate a duplicate subset.

C++

```
1 // LC 90 - Subsets II (with duplicates)
2 vector<vector<int>> subsetsWithDup(vector<int>& nums) {
3     sort(nums.begin(), nums.end()); // MUST sort first
4     vector<vector<int>> result;
5     vector<int> current;
6     function<void(int)> bt = [&](int start) {
7         result.push_back(current);
8         for (int i = start; i < (int)nums.size(); i++) {
9             // Skip duplicate values at the SAME level
10            if (i > start && nums[i] == nums[i-1]) continue;
11            current.push_back(nums[i]);
12            bt(i + 1);
13            current.pop_back();
14        }
15    };
16    bt(0);
17    return result;
18 }
19 // The condition i > start (NOT i > 0) is crucial:
20 // It only skips duplicates at the SAME recursive level
21 // i > 0 would incorrectly skip valid elements in deeper levels
```

3 Permutations

3.1 Permutations I — Distinct Elements (LC 46)

Generate all $n!$ permutations of distinct integers. Use a `used[]` boolean array to track which elements are in the current permutation. At each position, try all unused elements.

```
C++
1 // LC 46 - Permutations
2 vector<vector<int>> permute(vector<int>& nums) {
3     int n = nums.size();
4     vector<vector<int>> result;
5     vector<int> current;
6     vector<bool> used(n, false);
7     function<void()> bt = [&]() {
8         if ((int)current.size() == n) {
9             result.push_back(current); return;
10        }
11        for (int i = 0; i < n; i++) {
12            if (used[i]) continue;
13            used[i] = true;           // CHOOSE
14            current.push_back(nums[i]);
15            bt();                     // EXPLORE
16            current.pop_back();       // UNCHOOSE
17            used[i] = false;
18        }
19    };
20    bt();
21    return result;
22 }
23 // Time: O(n * n!)   Space: O(n)
24
25 // Alternative: swap-based (no used[] array)
26 void btSwap(vector<int>& nums, int start, vector<vector<int>>& res) {
27     if (start == (int)nums.size()) { res.push_back(nums); return; }
28     for (int i = start; i < (int)nums.size(); i++) {
29         swap(nums[start], nums[i]);
30         btSwap(nums, start+1, res);
31         swap(nums[start], nums[i]); // UNCHOOSE: restore
32     }
33 }
```

3.2 Permutations II — With Duplicates (LC 47)

Sort first. Skip a duplicate element if its previous identical element has NOT been used (meaning it was already chosen at this level and we'd generate the same permutation). Condition: `!used[i-1] && nums[i] == nums[i-1]`.

C++

```
1 // LC 47 - Permutations II (with duplicates)
2 vector<vector<int>> permuteUnique(vector<int>& nums) {
3     sort(nums.begin(), nums.end());
4     int n = nums.size();
5     vector<vector<int>> result;
6     vector<int> current;
7     vector<bool> used(n, false);
8     function<void()> bt = [&]() {
9         if ((int)current.size() == n) { result.push_back(current); return; }
10        for (int i = 0; i < n; i++) {
11            if (used[i]) continue;
12            // Skip: same value as previous AND previous was NOT used
13            // (means we already explored this choice at this level)
14            if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue;
15            used[i] = true; current.push_back(nums[i]);
16            bt();
17            current.pop_back(); used[i] = false;
18        }
19    };
20    bt();
21    return result;
22 }
23 // The !used[i-1] condition ensures we only use the first occurrence
24 // of each duplicate value at each recursive level
```


4 Combinations

4.1 Combination Sum I — Unlimited Reuse (LC 39)

Find all combinations of candidates that sum to target. Each number can be used unlimited times. Pass the same index i (not $i+1$) to allow reuse. Prune if remaining target < 0 .

```
C++
1 // LC 39 - Combination Sum (unlimited reuse, distinct candidates)
2 vector<vector<int>> combinationSum(vector<int>& cans, int target) {
3     sort(cans.begin(), cans.end()); // sort for pruning
4     vector<vector<int>> result;
5     vector<int> current;
6     function<void(int,int)> bt = [&](int start, int rem) {
7         if (rem == 0) { result.push_back(current); return; }
8         for (int i = start; i < (int)cans.size(); i++) {
9             if (cans[i] > rem) break; // PRUNE: sorted, all larger won't work
10            current.push_back(cans[i]);
11            bt(i, rem - cans[i]); // i (not i+1): allow reuse
12            current.pop_back();
13        }
14    };
15    bt(0, target);
16    return result;
17 }
18 // Time: O(n^(T/M)) where T=target, M=min candidate Space: O(T/M)
```

4.2 Combination Sum II — Each Number Used Once (LC 40)

Each number used at most once. Has duplicates. Sort + skip same value at same level. Pass $i+1$ (not i) to prevent reuse.

```
C++
1 // LC 40 - Combination Sum II (each used once, duplicates in input)
2 vector<vector<int>> combinationSum2(vector<int>& cans, int target) {
3     sort(cans.begin(), cans.end());
4     vector<vector<int>> result;
5     vector<int> current;
6     function<void(int,int)> bt = [&](int start, int rem) {
7         if (rem == 0) { result.push_back(current); return; }
8         for (int i = start; i < (int)cans.size(); i++) {
9             if (cans[i] > rem) break;
10            // Skip duplicates at same level
11            if (i > start && cans[i] == cans[i-1]) continue;
12            current.push_back(cans[i]);
13            bt(i + 1, rem - cans[i]); // i+1: no reuse
14            current.pop_back();
15        }
16    };
17    bt(0, target);
18    return result;
19 }
```

4.3 Letter Combinations of Phone Number (LC 17)

C++

```
1 // LC 17 - Letter Combinations of a Phone Number
2 vector<string> letterCombinations(string digits) {
3     if (digits.empty()) return {};
4     vector<string> phone = {"", "", "abc", "def", "ghi", "jkl",
5                             "mno", "pqrs", "tuv", "wxyz"};
6     vector<string> result;
7     string current = "";
8     function<void(int)> bt = [&](int pos) {
9         if (pos == (int)digits.size()) { result.push_back(current); return; }
10        for (char ch : phone[digits[pos]-'0']) {
11            current += ch; // CHOOSE
12            bt(pos + 1);   // EXPLORE
13            current.pop_back(); // UNCHOOSE
14        }
15    };
16    bt(0);
17    return result;
18 }
19 // Time: O(4^n * n) n=digits length (max 4 letters per digit)
```

5 Palindrome Partitioning

5.1 All Palindrome Partitions (LC 131) — Medium

Partition string s into substrings where every substring is a palindrome. Precompute $\text{isPalin}[i][j]$ to check palindromes in $O(1)$ during backtracking. At each position, try all valid palindrome endings.

```
C++
1 // LC 131 - Palindrome Partitioning
2 vector<vector<string>> partition(string s) {
3     int n = s.size();
4     // Precompute palindrome table -  $O(n^2)$ 
5     vector<vector<bool>> isPalin(n, vector<bool>(n, false));
6     for (int i=n-1; i>=0; i--)
7         for (int j=i; j<n; j++)
8             isPalin[i][j]=(s[i]==s[j])&&(j-i<=2||isPalin[i+1][j-1]);
9     vector<vector<string>> result;
10    vector<string> current;
11    function<void(int)> bt = [&](int start) {
12        if (start == n) { result.push_back(current); return; }
13        for (int end = start; end < n; end++) {
14            if (isPalin[start][end]) { // PRUNE: only palindromes
15                current.push_back(s.substr(start, end-start+1));
16                bt(end + 1);
17                current.pop_back();
18            }
19        }
20    };
21    bt(0);
22    return result;
23 }
24 // Time:  $O(n * 2^n)$  Space:  $O(n^2)$  for palindrome table
```

6

N-Queens & N-Queens II (LC 51, 52)

6.1 N-Queens — Place n queens on n×n board with no attacks

Place one queen per row. Track which columns, main diagonals (row-col), and anti-diagonals (row+col) are occupied.

Key insight: for the main diagonal, row-col is constant; for anti-diagonal, row+col is constant. Use bitmasks or sets for O(1) conflict checking.

```

C++
1 // LC 51 - N-Queens: return all valid board configurations
2 vector<vector<string>> solveNQueens(int n) {
3     vector<vector<string>> result;
4     vector<int> queens(n, -1); // queens[row] = col
5     set<int> cols, diag1, diag2; // diag1=row-col, diag2=row+col
6     function<void(int)> bt = [&](int row) {
7         if (row == n) {
8             // Build board from queens[]
9             vector<string> board(n, string(n, '.'));
10            for (int r=0; r<n; r++) board[r][queens[r]]='Q';
11            result.push_back(board);
12            return;
13        }
14        for (int col = 0; col < n; col++) {
15            if (cols.count(col) || diag1.count(row-col) || diag2.count(row+col))
16                continue; // PRUNE: under attack
17            queens[row] = col;
18            cols.insert(col); diag1.insert(row-col); diag2.insert(row+col);
19            bt(row + 1);
20            cols.erase(col); diag1.erase(row-col); diag2.erase(row+col);
21        }
22    };
23    bt(0);
24    return result;
25 }
26 // Time: O(n!) Space: O(n)

```

6.2 N-Queens — Bitmask Optimisation (LC 52)

Use integer bitmasks instead of sets for O(1) bitwise column/diagonal checks. Available positions = ~(cols | diag1 | diag2) & fullMask.

C++

```
1 // LC 52 - N-Queens II: count solutions only (bitmask approach)
2 int totalNQueens(int n) {
3     int count = 0;
4     int full = (1<<n)-1; // all n bits set = all columns available
5     function<void(int,int,int)> bt = [&](int cols, int d1, int d2) {
6         if (cols == full) { count++; return; }
7         // Available positions: columns not attacked
8         int avail = ~(cols|d1|d2) & full;
9         while (avail) {
10             int pos = avail & (-avail); // lowest set bit = rightmost available col
11             avail &= avail-1; // clear that bit
12             bt(cols|pos, (d1|pos)<<1, (d2|pos)>>1);
13             // new cols new main diag new anti diag
14         }
15     };
16     bt(0, 0, 0);
17     return count;
18 }
19 // Time: O(n!) Space: O(n) - much faster in practice than set-based
```

7 Sudoku Solver (LC 37) — Hard

7.1 Constraint Modelling

Maintain bitmasks for each row (9 bits), each column (9 bits), and each 3x3 box (9 bits). Bit i is set if digit $i+1$ is already placed. Available digits for cell $(r,c) = \sim(\text{rowMask}[r] \mid \text{colMask}[c] \mid \text{boxMask}[r/3*3+c/3]) \& 0x1FF$.

```

C++
1 // LC 37 - Sudoku Solver
2 void solveSudoku(vector<vector<char>>& board) {
3     int rowM[9]={}, colM[9]={}, boxM[9]={};
4     vector<pair<int,int>> empty;
5     // Initialise masks from existing digits
6     for (int r=0;r<9;r++) for (int c=0;c<9;c++) {
7         if (board[r][c]!='.') { empty.push_back({r,c}); continue; }
8         int bit = 1<<(board[r][c]-'1');
9         rowM[r]|=bit; colM[c]|=bit; boxM[r/3*3+c/3]|=bit;
10    }
11    function<bool(int)> bt = [&](int idx) -> bool {
12        if (idx==(int)empty.size()) return true;
13        auto [r,c] = empty[idx];
14        int avail = ~(rowM[r]|colM[c]|boxM[r/3*3+c/3]) & 0x1FF;
15        while (avail) {
16            int bit = avail & (-avail); // lowest available digit bit
17            avail &= avail-1;
18            int d = __builtin_ctz(bit); // digit index (0-based)
19            board[r][c] = '1'+d;
20            rowM[r]|=bit; colM[c]|=bit; boxM[r/3*3+c/3]|=bit;
21            if (bt(idx+1)) return true; // solved!
22            rowM[r]^=bit; colM[c]^=bit; boxM[r/3*3+c/3]^=bit;
23            board[r][c]='.';
24        }
25        return false;
26    };
27    bt(0);
28 }
29 // Time: O(9^empty_cells) worst case   Space: O(1) extra

```

8 Word Search on Grid

8.1 Word Search I (LC 79) — Medium

Search for a word in a 2D grid — can move in 4 directions, no reuse. DFS from each cell. Mark cell visited during recursion, restore after.

```

C++
1 // LC 79 - Word Search
2 bool exist(vector<vector<char>>& board, string word) {
3     int m=board.size(), n=board[0].size();
4     int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
5     function<bool(int,int,int)> dfs = [&](int r, int c, int k) -> bool {
6         if (k == (int)word.size()) return true; // all chars matched
7         if (r<0||r>=m||c<0||c>=n||board[r][c]!=word[k]) return false;
8         char orig = board[r][c];
9         board[r][c] = '#'; // mark visited
10        for (int d=0;d<4;d++)
11            if (dfs(r+dr[d], c+dc[d], k+1)) { board[r][c]=orig; return true; }
12        board[r][c] = orig; // restore
13        return false;
14    };
15    for (int r=0;r<m;r++) for (int c=0;c<n;c++)
16        if (dfs(r,c,0)) return true;
17    return false;
18 }
19 // Time: O(m*n * 4^L) L=word length Space: O(L) recursion

```

8.2 Word Search II — Multiple Words via Trie (LC 212) — Hard

Find all words from a dictionary that exist in the board. Naive approach: run Word Search I for each word $\rightarrow O(\text{words} * m * n * 4^L)$. Trie approach: build Trie of all words, then DFS using Trie to guide search. Remove found words from Trie to avoid duplicates and prune exhausted branches.

C++

```

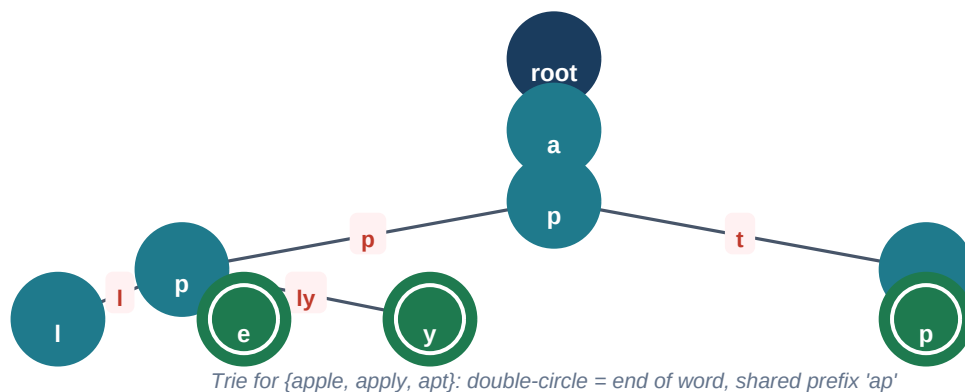
1  struct TrieNode {
2      TrieNode* ch[26]={};
3      string word = ""; // non-empty means a word ends here
4  };
5  vector<string> findWords(vector<vector<char>>& board,
6                          vector<string>& words) {
7      TrieNode* root = new TrieNode();
8      // Build trie from word list
9      for (auto& w : words) {
10         TrieNode* cur=root;
11         for (char c:w) { if(!cur->ch[c-'a']) cur->ch[c-'a']=new TrieNode();
12                         cur=cur->ch[c-'a']; }
13         cur->word=w;
14     }
15     int m=board.size(), n=board[0].size();
16     vector<string> result;
17     int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
18     function<void(int,int,TrieNode*)> dfs=[&](int r,int c,TrieNode* node){
19         if(r<0||r>=m||c<0||c>=n||board[r][c]!='#') return;
20         char ch=board[r][c];
21         TrieNode* next=node->ch[ch-'a'];
22         if(!next) return; // PRUNE: no word with this prefix
23         if(next->word!="") { // found a word
24             result.push_back(next->word);
25             next->word=""; // remove to avoid duplicates
26         }
27         board[r][c]='#'; // mark visited
28         for(int d=0;d<4;d++) dfs(r+dr[d],c+dc[d],next);
29         board[r][c]=ch; // restore
30     };
31     for(int r=0;r<m;r++) for(int c=0;c<n;c++) dfs(r,c,root);
32     return result;
33 }
34 // Time: O(m*n*4^L + W*L) W=#words, L=avg length Space: O(W*L)

```


9 Trie — Core Theory & Implementation

9.1 What is a Trie?

A **Trie** (prefix tree or digital tree) is a tree where each node represents a character, and paths from root to marked nodes spell out words. All strings sharing a common prefix share the same path in the Trie. This enables $O(L)$ insert, search, and prefix queries — where L is the string length — independent of the number of stored strings.



9.2 Full Trie Implementation (LC 208)

C++

```

1  // LC 208 — Implement Trie (Prefix Tree)
2  class Trie {
3      struct TrieNode {
4          TrieNode* children[26];
5          bool isEnd;
6          TrieNode() : isEnd(false) {
7              fill(children, children+26, nullptr);
8          }
9      };
10     TrieNode* root;
11 public:
12     Trie() { root = new TrieNode(); }
13
14     void insert(const string& word) {
15         TrieNode* cur = root;
16         for (char c : word) {
17             int idx = c - 'a';
18             if (!cur->children[idx])
19                 cur->children[idx] = new TrieNode();
20             cur = cur->children[idx];
21         }
22         cur->isEnd = true;
23     } // Time: O(L)   Space: O(L * 26) per word
24
25     bool search(const string& word) {
26         TrieNode* cur = root;
27         for (char c : word) {
28             int idx = c - 'a';
29             if (!cur->children[idx]) return false;
30             cur = cur->children[idx];
31         }
32         return cur->isEnd;
33     } // Time: O(L)
34
35     bool startsWith(const string& prefix) {
36         TrieNode* cur = root;
37         for (char c : prefix) {
38             if (!cur->children[c-'a']) return false;
39             cur = cur->children[c-'a'];
40         }
41         return true;
42     } // Time: O(L)
43
44     // Delete a word from the trie
45     bool remove(const string& word) {
46         function<bool(TrieNode*, int)> del = [&](TrieNode* n, int i) -> bool {
47             if (i == (int)word.size()) {
48                 if (!n->isEnd) return false;
49                 n->isEnd = false;
50                 // Can delete node if it has no children
51                 for (auto c : n->children) if (c) return false;
52                 return true;
53             }
54             int idx = word[i] - 'a';
55             if (!n->children[idx]) return false;
56             if (del(n->children[idx], i+1)) {
57                 delete n->children[idx];
58                 n->children[idx] = nullptr;
59                 // Can delete this node too if no other children
60                 if (!n->isEnd) {
61                     for (auto c : n->children) if (c) return false;
62                     return true;
63                 }
64             }
65             return false;
66         };
67         return del(root, 0);
68     }
69 };
70 // Space: O(ALPHABET_SIZE * L * W)  W=number of words

```

10 Trie — Classic Problems

10.1 Design Add and Search Words (LC 211) — Medium

Support wildcard '.' which matches any character. Normal characters traverse the Trie deterministically. For '.', try all 26 children recursively.

```

C++
1  class WordDictionary {
2      struct Node { Node* ch[26]={}; bool end=false; };
3      Node* root;
4  public:
5      WordDictionary() { root = new Node(); }
6      void addWord(string word) {
7          Node* cur=root;
8          for (char c:word) {
9              if(!cur->ch[c-'a']) cur->ch[c-'a']=new Node();
10             cur=cur->ch[c-'a'];
11         }
12         cur->end=true;
13     }
14     bool search(string word) {
15         function<bool(Node*,int)> dfs=[&](Node* n, int i)->bool{
16             if(i==(int)word.size()) return n->end;
17             char c=word[i];
18             if(c!='.') {
19                 return n->ch[c-'a'] && dfs(n->ch[c-'a'],i+1);
20             }
21             // Wildcard: try all children
22             for(int j=0;j<26;j++)
23                 if(n->ch[j] && dfs(n->ch[j],i+1)) return true;
24             return false;
25         };
26         return dfs(root,0);
27     }
28 };
29 // search: O(26^wildcards * L) worst case

```

10.2 Replace Words (LC 648) — Medium

Given a list of roots and a sentence, replace each word in the sentence with its shortest root. Build Trie from roots. For each word, traverse Trie to find shortest prefix.

C++

```

1  string replaceWords(vector<string>& dictionary, string sentence) {
2      // Build Trie from roots
3      struct Node { Node* ch[26]={}; bool isRoot=false; };
4      Node* root = new Node();
5      for (auto& r : dictionary) {
6          Node* cur=root;
7          for (char c:r) { if(!cur->ch[c-'a']) cur->ch[c-'a']=new Node();
8                          cur=cur->ch[c-'a']; }
9          cur->isRoot=true;
10     }
11     // Replace each word
12     istringstream iss(sentence);
13     string result, word;
14     while (iss >> word) {
15         if (!result.empty()) result += ' ';
16         Node* cur=root; string prefix="";
17         bool replaced=false;
18         for (char c:word) {
19             if(!cur->ch[c-'a']) break;
20             cur=cur->ch[c-'a']; prefix+=c;
21             if(cur->isRoot) { result+=prefix; replaced=true; break; }
22         }
23         if(!replaced) result+=word;
24     }
25     return result;
26 }
27 // Time: O(sum_roots + |sentence|)   Space: O(sum_root_lengths * 26)

```

10.3 Search Suggestions System (LC 1268) — Medium

After each character typed, suggest up to 3 products with the matching prefix, lexicographically sorted. Sort products first. For each prefix, find matching products using Trie or binary search.

C++

```

1  // Binary search approach - simpler and same complexity
2  vector<vector<string>> suggestedProducts(
3      vector<string>& products, string searchWord) {
4      sort(products.begin(), products.end());
5      vector<vector<string>> result;
6      string prefix = "";
7      for (char c : searchWord) {
8          prefix += c;
9          // Lower bound: first product >= prefix
10         auto lo = lower_bound(products.begin(), products.end(), prefix);
11         vector<string> suggestions;
12         for (int i=0; i<3 && lo!=products.end(); i++, lo++) {
13             // Check if still has this prefix
14             if (lo->substr(0, prefix.size()) == prefix)
15                 suggestions.push_back(*lo);
16             else break;
17         }
18         result.push_back(suggestions);
19     }
20     return result;
21 }
22 // Time: O(n log n + L * (log n + 3))   Space: O(L) for prefix string

```

11 Trie — Advanced Applications

11.1 Maximum XOR of Two Numbers (LC 421) — Medium

Find the maximum XOR of any two numbers in an array. Build a binary Trie of all numbers (bit by bit from MSB to LSB). For each number, greedily choose the opposite bit at each level to maximise XOR.

C++

```

1 // LC 421 - Maximum XOR of Two Numbers in an Array
2 struct XorTrieNode {
3     XorTrieNode* ch[2] = {};
4 };
5
6 int findMaximumXOR(vector<int>& nums) {
7     XorTrieNode* root = new XorTrieNode();
8     // Insert all numbers into binary Trie (bit 31 to bit 0)
9     for (int x : nums) {
10         XorTrieNode* cur = root;
11         for (int i=31; i>=0; i--) {
12             int bit = (x>>i) & 1;
13             if (!cur->ch[bit]) cur->ch[bit] = new XorTrieNode();
14             cur = cur->ch[bit];
15         }
16     }
17     // For each number, query maximum XOR
18     int ans = 0;
19     for (int x : nums) {
20         XorTrieNode* cur = root;
21         int xorVal = 0;
22         for (int i=31; i>=0; i--) {
23             int bit = (x>>i) & 1;
24             int want = 1-bit;           // want opposite bit for max XOR
25             if (cur->ch[want]) {
26                 xorVal |= (1<<i);       // this bit of XOR = 1
27                 cur = cur->ch[want];
28             } else {
29                 cur = cur->ch[bit];     // must take same bit
30             }
31         }
32         ans = max(ans, xorVal);
33     }
34     return ans;
35 }
36 // Time: O(n * 32)   Space: O(n * 32)

```

11.2 Map Sum Pairs (LC 677) — Medium

Insert key-value pairs. Find the sum of all values with keys starting with a given prefix. Store the sum at each node of the prefix that contributes to it.

C++

```
1 // LC 677 — Map Sum Pairs
2 class MapSum {
3     struct Node { Node* ch[26]={}; int val=0; };
4     Node* root;
5     unordered_map<string,int> map; // store inserted values
6 public:
7     MapSum() { root = new Node(); }
8     void insert(string key, int val) {
9         int delta = val - map[key]; // change in value (handles updates)
10        map[key] = val;
11        Node* cur = root;
12        for (char c : key) {
13            if (!cur->ch[c-'a']) cur->ch[c-'a'] = new Node();
14            cur = cur->ch[c-'a'];
15            cur->val += delta; // each node on path gets delta
16        }
17    }
18    int sum(string prefix) {
19        Node* cur = root;
20        for (char c : prefix) {
21            if (!cur->ch[c-'a']) return 0;
22            cur = cur->ch[c-'a'];
23        }
24        return cur->val; // sum of all values with this prefix
25    }
26 };
27 // insert: O(L)    sum: O(L)    Space: O(total_key_length * 26)
```

12 Complexity Cheat Sheet & LeetCode Map

12.1 Backtracking Complexity Summary

Problem Type	Time Complexity	Space	Pruning Key
Subsets of n elements	$O(n \cdot 2^n)$	$O(n)$	Sort + skip dup at same level
Permutations (distinct)	$O(n \cdot n!)$	$O(n)$	used[] array prevents reuse
Permutations (duplicates)	$O(n \cdot n!)$	$O(n)$	Sort + skip if !used[i-1] & dup
Combinations nCk	$O(k \cdot C(n,k))$	$O(k)$	start index avoids reversed dups
Combination Sum	$O(n^{(T/M)})$	$O(T/M)$	Sort + break if cand > remaining
N-Queens	$O(n!)$	$O(n)$	Bitmask $O(1)$ conflict check
Sudoku Solver	$O(9^{\text{empty}})$	$O(1)$	Bitmask available digits
Palindrome Partition	$O(n \cdot 2^n)$	$O(n^2)$	Precompute isPalin table
Word Search I	$O(m \cdot n \cdot 4^L)$	$O(L)$	In-place visited marking
Word Search II (Trie)	$O(m \cdot n \cdot 4^L)$	$O(W \cdot L)$	Trie guides DFS, prune no-prefix

Table 1: Backtracking complexity summary

12.2 Trie Complexity Summary

Operation	Time	Space	Notes
Insert word	$O(L)$	$O(L \cdot \Sigma)$	Σ = alphabet size (26 for lowercase)
Search word	$O(L)$	$O(1)$	Exact match
Prefix search	$O(L)$	$O(1)$	startsWith
Delete word	$O(L)$	$O(L)$	Recursive cleanup
Count words	$O(W \cdot L)$	$O(W \cdot L)$	Build then query
Wildcard search	$O(26^w \cdot L)$	$O(L)$	w = number of wildcards
XOR Trie insert	$O(32)$	$O(32 \cdot n)$	32-bit integers
XOR Trie query	$O(32)$	$O(1)$	Greedy opposite bit
Space (array)	—	$O(\Sigma \cdot W \cdot \text{avgL})$	Array of children pointers
Space (hashmap)	—	$O(W \cdot \text{avgL})$	HashMap children; slower access

Table 2: Trie operations complexity (L =word length, W =number of words)

12.3 Pattern Decision Guide

[illegible]

12.4 LeetCode Problem Map

#	Problem	Pattern	Diff
17	Letter Combinations of Phone Number	Backtracking	Medium
22	Generate Parentheses	Backtracking + validity	Medium
37	Sudoku Solver	Backtracking + bitmask	Hard
39	Combination Sum	BT + unlimited reuse	Medium
40	Combination Sum II	BT + skip duplicates	Medium
46	Permutations	BT + used[] array	Medium
47	Permutations II	BT + !used[i-1] skip	Medium
51	N-Queens	BT + diag sets	Hard
52	N-Queens II	BT + bitmask optimization	Hard
77	Combinations	BT with start index	Medium
78	Subsets	BT include/exclude	Medium
79	Word Search	DFS + backtrack in grid	Medium
90	Subsets II	BT + skip dup same level	Medium
93	Restore IP Addresses	BT + pruning	Medium
131	Palindrome Partitioning	BT + isPalin table	Medium
212	Word Search II	Trie + DFS backtracking	Hard
208	Implement Trie	Trie insert/search/prefix	Medium
211	Design Add and Search Words	Trie + DFS wildcard	Medium

421	Maximum XOR of Two Numbers in Array	Binary Trie	Medium
472	Concatenated Words	Trie + DP	Hard
588	Design In-Memory File System	Trie	Hard
648	Replace Words	Trie prefix replacement	Medium
677	Map Sum Pairs	Trie with prefix sum	Medium
1268	Search Suggestions System	Trie / Binary Search	Medium

Table 3: 23 LeetCode problems — Backtracking & Trie

■ Golden Rules — Backtracking & Trie

1. Backtracking = DFS on decision tree. CHOOSE → EXPLORE → UNCHOOSE. Always restore state.
2. Deduplication: SORT the input first. Skip if $i > \text{start}$ && $\text{nums}[i] == \text{nums}[i-1]$.
3. Permutations vs Combinations: Permutations use `used[]` (no start). Combinations use start (no `used[]`).
4. Combination Sum reuse: pass i (same index). No reuse: pass $i+1$.
5. N-Queens bitmask: $\text{avail} = \sim(\text{cols}|\text{d1}|\text{d2}) \& \text{full}$. Pick lowest bit: $\text{bit} = \text{avail} \& (-\text{avail})$.
6. Pruning is EVERYTHING in backtracking. Sort + break when candidate exceeds remaining = huge speedup.
7. Word Search II: build Trie from word list. DFS uses Trie node to guide search and prune dead branches.
8. Trie insert/search: $O(L)$ per operation. Never $O(n)$ — independent of number of stored words.
9. XOR Trie: build bit-by-bit from MSB. For max XOR, greedily choose opposite bit if available.
10. In-place visited marking (`board[r][c]='#'`): saves $O(m*n)$ extra space vs a separate visited array.